CLINICAL TRIAL ELIGIBILITY ENGINE

Full technical spec & pitch rehearsal guide

AI-powered patient-to-trial matching. Seconds, not weeks.

80% of trial timeline overruns caused by recruitment delays

Days avg manual screening time per patient

Seconds with this tool — for every eligible patient

Built at the AIxBio Hackathon · Bayer Co.Lab, Cambridge MA · Boston Tech Week 2026

DOCUMENT GUIDE

What's Inside

PART 1 — PRODUCT OVERVIEW	Problem, solution, and why it matters
PART 2 — HOW IT WORKS	The three-step pipeline, end to end
PART 3 — TECHNICAL ARCHITECTURE	Every file, module, and design decision
PART 4 — DATABASE DESIGN	Schema, caching strategy, and persistence
PART 5 — API REFERENCE	All endpoints with input/output shapes
PART 6 — KEY ENGINEERING DECISIONS	The 'why' behind dual-pass matching and caching
PART 7 — PITCH REHEARSAL NOTES	Talking points, Q&A; prep, and the demo script

PART 1

Product Overview

What is this, who is it for, and why does it matter?

The Problem

Finding the right clinical trial for a patient is one of the most time-consuming jobs in medical research. A clinical trial coordinator reads through a patient's entire medical history, then reads through the eligibility requirements of dozens of trials — and matches them manually, line by line. Most eligible patients are never found — not because the trials don't exist, but because the process doesn't scale.

80% of clinical trial timeline overruns are caused by recruitment delays. The average manual screening time per patient is measured in days. This creates a compounding problem: patients miss trials they qualify for, and trials run slow or fail to recruit entirely.

The Solution

The Clinical Trial Eligibility Engine automates the matching process using AI. A coordinator pastes or loads a patient's health record, selects a condition to search against, and hits Screen. Within seconds — not days — they get a ranked list of currently recruiting trials, each with a criterion-by-criterion eligibility breakdown they can review before taking any action.

FAST
Screens a patient against 8–20 live trials in seconds. Parallel AI calls with bounded concurrency.

TRANSPARENT
Every eligibility decision is shown, explained, and auditable. No black boxes.

SAFE
The system never marks a patient ineligible due to missing data. Uncertainty is surfaced, not buried.

Who Uses This

The primary user is a clinical trial coordinator or research nurse who manages patient pipelines for a hospital or research site. Secondary users include patients or advocates who want to find trials they might qualify for. The UI is designed to be readable by both — technical outputs are always paired with a plain-language summary.

What It Is Not

This is a decision-support tool, not a decision-making tool. Every AI verdict is intended to be reviewed by a qualified human coordinator before any action is taken. The system does not enroll patients, does not store real patient data, and does not make clinical recommendations.

PART 2

How It Works

The three-step pipeline from patient record to ranked trial list.

01

PASTE

Input

The coordinator pastes or loads a patient's health notes. A synthetic example can be generated on-demand with one click.

02

EXTRACT

AI reads the record

The LLM parses the raw notes into a structured patient profile: age, diagnoses, medications, lab values, ECOG status, and more. This profile drives all downstream matching.

03

CONFIRM

Human checkpoint

The extracted primary condition is shown to the user for review. They can edit it before the search runs — ensuring the trial search is anchored to the right disease area.

04

SEARCH

Live trial data

ClinicalTrials.gov is queried in real time for currently recruiting trials matching the condition. Up to 20 trials are returned with their full eligibility criteria text.

05

MATCH

Dual-pass AI scoring

Each trial is scored concurrently. Two LLM passes run at different temperatures — their verdicts are reconciled, with disagreements marked UNCERTAIN. Results are ranked by overall match score.

06

REVIEW

Criterion breakdown

The coordinator sees a ranked queue of trials. Clicking any trial shows the full criterion-by-criterion breakdown: ELIGIBLE, INELIGIBLE, or UNCERTAIN — with a plain-English reason for each.

KEY DEMO MOMENT

The most important moment in the demo is Step 6 — the criterion breakdown. It shows that every AI decision is fully auditable. That transparency is what separates this from a chatbot, and what makes it trustworthy in a clinical setting.

PART 3

Technical Architecture

Every file, module, and the role it plays in the system.

Tech Stack

LAYER	TECHNOLOGY	NOTES
Frontend	React + Vite	5-step wizard UI, inline CSS vars for theming
Styling	Tailwind CSS	Utility classes + CSS custom properties
Backend	Python 3.11 + FastAPI	Async throughout; CORS enabled for local dev
AI / LLM	Nebius API — Llama 3.3 70B Instruct	OpenAI-compatible SDK; temperature 0.1 for JSON stability
Database	Supabase (PostgreSQL)	3 tables; RLS enabled; content-hash dedup
Trial Data	ClinicalTrials.gov v2 API	Live search; only RECRUITING trials returned

Backend Modules (Python/FastAPI)

The backend is a single FastAPI application. Each Python file has a single, clear responsibility. There is no bloat — every file is short and does exactly one thing.

`main.py`	FastAPI app entry point. Registers all routes, configures CORS, and orchestrates the /match pipeline. Contains the background task that caches parsed trial criteria after the response is sent.
`extractor.py`	All LLM extraction logic. Two functions: extract_patient_profile() reads raw patient text and returns a structured PatientProfile. parse_trial_criteria() takes raw eligibility text and returns a structured list of inclusion/exclusion criteria. Both call Llama 3.3 70B via Nebius.
`matcher.py`	Per-criterion matching logic and bulk orchestration. match_patient_to_trial() runs two concurrent LLM passes and reconciles disagreements. bulk_match() fans out across all trials using asyncio.gather() with a bounded semaphore.

`trials_fetcher.py`	ClinicalTrials.gov v2 API integration. Searches for RECRUITING trials by condition and parses the response into TrialResult objects. Returns raw eligibility criteria text for each trial.
`models.py`	All Pydantic schemas. Every LLM response is validated through a Pydantic model before touching the database. PatientProfile, TrialResult, MatchOutput, CriterionVerdict, and all API request/response shapes live here.
`persistence.py`	Supabase read/write helpers. Handles patient deduplication (SHA-256 content hash), match result caching, trial upserts, and queue retrieval. Gracefully falls back to in-memory if the database is unavailable.
`db.py`	Single-line Supabase client initialization. Reads credentials from environment variables.

Frontend Components (React/Vite)

The frontend is a single-page React application with a 5-step state machine. Each step is a distinct component. State lives entirely in App.jsx — components are stateless and receive everything via props.

`App.jsx`	Root component and state machine. Manages the 5 steps: input → confirm → screening → results → detail. Holds all application state including extracted profile, match results, selected result, and error state. Contains mock data for offline demo mode.
`PatientInput.jsx`	Step 1 UI. Textarea for pasting patient notes, plus a button to generate a synthetic patient via the LLM. Shows a loading spinner during extraction.
`ScreeningLoader.jsx`	Step 3 UI. Animated loading state shown while the AI is screening. Displays the condition being searched and transitions smoothly when done.
`Queue.jsx`	Step 4 UI. Ranked list of match results. Each card shows trial title, NCT ID, eligibility status badge, and overall match score. Clicking a card navigates to the detail view.
`MatchResult.jsx`	Step 5 UI. Full criterion-by-criterion breakdown for a selected trial. Shows each criterion with a colored verdict badge (ELIGIBLE / INELIGIBLE / UNCERTAIN) and a one-sentence reason. Includes the plain-language summary at the top.
`api.js`	All fetch() calls centralized here. Components never call fetch() directly. Handles base URL configuration, JSON parsing, and error extraction from the FastAPI response shape.

PART 4

Database Design

Three tables, content-hash deduplication, and a smart caching layer.

Schema Overview

The database has exactly three tables. The design is deliberately minimal — the goal is to persist results across sessions and enable instant replays, not to model a full clinical data system.

patients

id	uuid	Primary key. Auto-generated UUID.
content_hash	text (unique)	SHA-256 of normalized raw text. Prevents duplicate patients when the same notes are submitted twice.
raw_text	text	Original health notes as pasted by the user.
extracted_profile	jsonb	Structured patient profile produced by the extraction LLM. Stored as JSONB for flexible querying.
created_at	timestamptz	Row creation timestamp.

trials

id	uuid	Primary key.
nct_id	text (unique)	ClinicalTrials.gov identifier (e.g. NCT05765812). Used as the upsert key.
title	text	Trial title from ClinicalTrials.gov.
condition	text	Disease area / condition from the trial record.
raw_criteria	text	Raw eligibility criteria text from ClinicalTrials.gov.
parsed_criteria	jsonb	AI-parsed inclusion/exclusion criteria list. Populated asynchronously in the background after /match responds.

match_results

id	uuid	Primary key.
patient_id	uuid (FK)	References patients.id. Cascades on delete.
trial_id	uuid (FK)	References trials.id. Cascades on delete.
overall_score	int (0–100)	AI-computed eligibility score. Used to rank the queue.

status	text	LIKELY_ELIGIBLE | LIKELY_INELIGIBLE | UNCERTAIN
criteria_breakdown	jsonb	Array of CriterionVerdict objects — the per-criterion audit trail.
summary	text	2–3 sentence plain-language summary for the coordinator.

Caching Strategy

The system uses two distinct caching mechanisms. Together they make demo replays feel instant and eliminate wasteful LLM re-runs in production.

	Before inserting a new patient row, the raw notes are SHA-256 hashe row with the same hash already exists, the existing patient_id is reuse patient twice never creates duplicate records.
	When a patient already has match results in the database for a given Supabase instead of calling the LLM again. The bulk_match() function skips any trial already in cache. This is what makes demo replays feel
t	After /match returns its response, a background asyncio task parses t screened trial and caches the structured JSON in the trials table. This never delays the demo.

PART 5

API Reference

All endpoints, inputs, and outputs. Base URL: http://localhost:8000

POST /extract

Summary	Extract a structured patient profile from raw health notes. Used for the confirmation step before matching.
Input	`{ patient_text: string }`
Output	`{ extracted_profile: PatientProfile }`
Notes	LLM call only — no database writes. Returns structured JSON including diagnoses, medications, lab values, and ECOG status.

POST /match

Summary	Run the full pipeline: extract patient profile, search ClinicalTrials.gov, dual-pass AI match across all trials, persist to Supabase, return top 10 results ranked by score.
Input	`{ patient_text, confirmed_condition, extracted_profile (optional), max_trials (default 8) }`
Output	`BulkMatchResponse: { patient_id, extracted_profile, total_screened, results[], persisted_to_db, cache_hits, llm_calls }`
Notes	Most complex endpoint. Handles cache hits, graceful DB fallback, and background criteria enrichment. The extracted_profile field avoids a duplicate LLM extraction call if /extract was already called.

POST /generate-sample

Summary	Generate a synthetic patient health summary using the LLM. Returns a realistic but fictional patient record for a randomly selected condition.
Input	`No body required.`
Output	`{ patient_text: string }`
Notes	Useful for demos. Uses temperature 0.9 for variety. Conditions include Fabry disease, ALS, CLL, Crohn's, lupus, and others.

GET /trials/search

Summary	Search ClinicalTrials.gov directly by condition or keyword. Returns trial metadata and raw eligibility criteria text.
Input	`Query params: condition (string), max_results (int, default 20)`
Output	`{ condition, count, trials[] }`
Notes	Only returns RECRUITING trials. Filters out trials with missing NCT ID or empty eligibility criteria.

GET /queue

Summary	Return all match results from Supabase, ranked by overall score descending. Powers the persistent queue view.
Input	`No parameters.`
Output	`{ count, results[] } — each result includes trial metadata and overall score.`
Notes	Joins match_results with trials to include nct_id, title, and condition. Limit 50 rows.

GET /queue/{id}

Summary	Return a single match result with full criterion breakdown and linked trial details.
Input	`Path param: result UUID`
Output	`Full match_results row joined with trials data.`
Notes	Used when the coordinator clicks a result to see the detailed eligibility breakdown.

GET /health

Summary	API and database readiness check. Probes all three Supabase tables.
Input	`No parameters.`
Output	`{ status: "ok", database: { ready, tables, errors } }`
Notes	Useful for pre-demo verification.

PART 6

Key Engineering Decisions

The 'why' behind each significant technical choice.

Dual-pass LLM matching with reconciliation

What: Every patient-trial match runs two concurrent LLM calls: one at temperature 0.1 (deterministic) and one at 0.5 (slightly more varied). The two verdicts are then reconciled criterion by criterion.

Why: A single LLM pass can hallucinate a confident ELIGIBLE or INELIGIBLE on ambiguous criteria. Running two passes and checking for disagreement surfaces the uncertainty explicitly. Criteria where the two passes disagree are downgraded to UNCERTAIN and flagged as 'contested'. This catches cases where the model is uncertain even if it doesn't say so.

Tradeoff: 2x LLM cost per trial. Mitigated by the caching layer — the cost is paid once per patient/trial pair, then never again.

UNCERTAIN preferred over INELIGIBLE for missing data

What: The matching prompt explicitly instructs the LLM: 'ALWAYS prefer UNCERTAIN over INELIGIBLE when data is simply missing.' A patient record that doesn't mention a lab value is UNCERTAIN on criteria that require that value — not INELIGIBLE.

Why: In clinical screening, missing information is not the same as a failed criterion. Marking a patient INELIGIBLE because their notes don't mention a specific test would eliminate patients who simply need that test ordered. The tool should surface gaps in information, not make decisions based on them.

Tradeoff: UNCERTAIN results require follow-up work by the coordinator. This is intentional — it's a flag to gather more data, not a dead end.

Pydantic validation on all LLM outputs

What: Every LLM response is parsed and validated through a Pydantic model before any downstream processing or database writes. A _parse_llm_json() helper strips markdown fences and extracts JSON from mixed-content responses.

Why: LLMs occasionally return malformed JSON, add prose before or after the JSON block, or wrap output in markdown code fences. Without a parsing layer, a single bad response would crash the entire pipeline. With Pydantic validation, a malformed response raises a clear exception and the trial is skipped — not the whole match run.

Tradeoff: Slightly more code. Worth it for production reliability.

Async throughout with bounded concurrency

What: The entire backend is async (FastAPI + asyncio + httpx). bulk_match() uses asyncio.gather() to fan out across all trials concurrently, with an asyncio.Semaphore(8) to cap simultaneous LLM calls.

Why: Matching 8 trials serially would take 8x as long. Concurrency makes the demo fast. The semaphore prevents rate-limit errors and keeps cost predictable.

Tradeoff: Async code is harder to debug than sync. For a hackathon this is an acceptable complexity tradeoff.

Nebius / Llama over GPT-4

What: All LLM calls use the Nebius-hosted Llama 3.3 70B Instruct model via an OpenAI-compatible endpoint. The client is literally OpenAI's Python SDK pointed at a different base_url.

Why: Compute for the hackathon was provided by Nebius. Llama 3.3 70B is competitive with GPT-4o on structured JSON extraction tasks. The OpenAI-compatible API means the model can be swapped out by changing two environment variables — no code changes needed.

Tradeoff: Slightly different JSON reliability characteristics vs. GPT-4. Handled by the Pydantic validation layer.

Graceful DB fallback to in-memory

What: If Supabase is unavailable during /match, results are returned in-memory using temporary UUIDs. The coordinator still gets their screening results — they're just not persisted.

Why: For a demo, database availability should never block the core functionality. The pipeline should work even if Supabase credentials are misconfigured.

Tradeoff: In-memory results are lost when the backend restarts. This is acceptable in demo context.

PART 7

Pitch Rehearsal Notes

Talking points, the demo script, anticipated questions, and what to emphasize for different audiences.

The One-Sentence Pitch

SAY THIS FIRST

We built an AI tool that matches patients to clinical trials in seconds — automating a process that today takes coordinators days to weeks by hand.

The Core Narrative (2 minutes)

Start with the problem, not the product. Lead with the human cost:

Beat 1 — The problem	80% of clinical trial timeline overruns are caused by recruitment delays. The average time to manually screen a single patient is measured in days. That's a coordinator reading a full medical history, then reading eligibility criteria for 20 trials, line by line. Most eligible patients are never connected to a trial — not because the trials don't exist, but because the process doesn't scale.
Beat 2 — The moment of insight	The insight is: this is fundamentally an information extraction and matching problem. The eligibility criteria are structured text. The patient record is structured text. An LLM can read both and tell you if they match — and it can do it for 20 trials at once, in seconds.
Beat 3 — The product	So we built this. You paste a patient record, confirm the condition, and hit Screen. The AI screens the patient against every currently recruiting trial on ClinicalTrials.gov for that condition — and gives you a ranked list with a full explanation of exactly which criteria were met, which were not, and which we don't have enough information to evaluate.
Beat 4 — The differentiator	The differentiator is the criterion breakdown. This is not a chatbot that says 'this patient might qualify.' Every decision is shown, explained, and auditable. That's the central trust problem in medical AI — and that's what we solved.
Beat 5 — The scope	Built in 24 hours at the AIxBio Hackathon at Bayer Co.Lab. Real data from ClinicalTrials.gov. Real AI. Fully functional from day one.

The Demo Script

1. Load patient	Click 'Generate Sample' to generate a synthetic patient. Say: 'This generates a realistic but fully fictional patient record — no real data anywhere in the system.'
2. Hit Analyze	Click Analyze. While it extracts, say: 'The AI is reading the notes and pulling out the structured facts — diagnoses, medications, lab values, ECOG status.'
3. Confirm condition	Show the extracted profile card. Say: 'The AI found the primary condition with high confidence. A coordinator can edit this before the search — this is the human checkpoint.'
4. Run Screening	Click Run Screening. While it loads, say: 'It's hitting ClinicalTrials.gov live right now, pulling every currently recruiting trial for this condition, and screening the patient against each one in parallel.'
5. Show the queue	When results appear, say: 'That's the queue. Ranked by match score. The coordinator's to-do list, built automatically.'
6. Click a result — THIS IS THE KEY MOMENT	Open the criterion breakdown. Say: 'This is what matters. Every criterion. Every verdict. Every reason — in plain English. The coordinator can see exactly why the AI said yes, no, or uncertain. Nothing is hidden.'
7. Point out UNCERTAIN	Find an UNCERTAIN criterion and say: 'This one is marked uncertain because the patient's notes don't mention it. That's not a failed criterion — it's a flag to go gather more information. Missing data is not a failure.'

Anticipated Questions — Q&A; Prep

Q: Is this using real patient data?

No. All patient data in the demo is fully synthetic — generated by the LLM. No real patient records are used, stored, or referenced anywhere. The tool is a decision-support system designed to be used with real data by real coordinators — but for the demo, everything is fictional.

Q: What if the AI gets it wrong?

That's why every result includes the full criterion breakdown with reasons. The AI's job is to do the first-pass read and surface the cases worth a coordinator's attention. The coordinator reviews every result before any action is taken. The tool reduces the time to screen — it doesn't replace the judgment that screening requires.

Q: How accurate is the matching?

We haven't run formal accuracy benchmarks — this is a hackathon build. What we can say is that the dual-pass reconciliation design specifically targets the main failure mode: overconfident verdicts on ambiguous data. Any criterion where the two LLM passes disagree is automatically downgraded to UNCERTAIN.

Q: Why Llama instead of GPT-4?

Compute was provided by Nebius for the hackathon. Llama 3.3 70B is highly capable on structured JSON tasks, and the OpenAI-compatible API means we can swap models by changing an environment variable. The architecture is model-agnostic.

Q: How does this scale?

The async parallel architecture handles 8–20 trials concurrently today. Scaling to larger trial sets is a matter of increasing the semaphore limit and compute. The caching layer means repeat screenings of the same patient are essentially free.

Q: What about HIPAA / data privacy?

This is a hackathon prototype. In production, patient data would need to be handled under a BAA with the LLM provider, data would be de-identified before any LLM call, and the database would be behind proper auth. Those are solvable problems — none of them require changes to the core matching logic.

Q: What's the next step after the hackathon?

The immediate next steps are: (1) a proper evaluation framework comparing AI verdicts against manual coordinator decisions, (2) user research with actual clinical trial coordinators, and (3) integration with EHR systems to pull patient data rather than requiring copy-paste.

Audience-Specific Emphasis

FOR INVESTORS / BUSINESS AUDIENCE	FOR TECHNICAL / ENGINEER AUDIENCE
• Lead with the 80% statistic on trial overruns • Emphasize that eligible patients are routinely missed — this is a human cost, not just an efficiency problem • Frame the criterion breakdown as the trust layer that makes AI adoptable in clinical settings • Mention that the pipeline is fully functional with real ClinicalTrials.gov data • Point to the hackathon context — fully functional in 24 hours	• Lead with the dual-pass reconciliation design — it's the interesting engineering choice • Explain the Pydantic validation layer and why it exists • Highlight the async parallel architecture with bounded semaphore • Show the content-hash dedup and caching strategy • Note the OpenAI-compatible Nebius endpoint — model-agnostic by design

CLOSING MOVE

The criterion breakdown is always the close. Whatever audience you're pitching to, end on that screen. Let them read one result. Let them see exactly what the AI said and why. That moment speaks louder than any slide.

Built in collaboration with C10 Labs, LabCentral, Bayer Co.Lab, and Evolved Technology. Compute provided by Nebius. Part of Boston Tech Week 2026.